Representation of Spatial Objects

A scientific report on the theoretical foundations and an experimental study of modern formats

Spatial Databases | Group 1 | Chapter 2 | November 2, 2026

Abstract

This report systematizes how spatial objects are represented, from entity and field models to raster, vector, half-planes, and spaghetti, network and topological structures. The main theoretical source is Chapter 2 “Representation of Spatial Objects” (pp. 29–67) of the book Spatial Databases: With Application to GIS by Rigaux, Scholl and Voisard (2002), checked against original works, TIGER/Line and the GeoJSON, TopoJSON, KML, Vector Tiles and GeoParquet specifications. Three experiments were run on synthetic data: shared boundaries and updates; complete processing of 100–10,000 polygons; and MVT delivery by zoom level. At 10,000 polygons, TopoJSON reduces file size by 33.70% compared with GeoJSON but has a higher build cost. GeoParquet ZSTD reads faster than GeoJSON at this scale, while the order reverses at 100 objects. MVT reduces the data needed for a viewport as zoom increases, while exhibiting measured quantization error. The results show that the choice must be tied to the information to be preserved, the scale and the task. The measurements are descriptive statistics of the published pipelines, not a universal ranking of formats. Code, data and per-repetition results are provided for reproduction.

Keywords spatial representation; entity and field models; topology; GeoJSON; TopoJSON; KML; vector tiles; GeoParquet.

1 Introduction

Representing spatial objects is the step from geographic phenomena to data that a system can store, query and exchange. The same area can be modeled as identified land parcels, an elevation field, a transportation network or a partition into regions. The choice of representation determines which information is retained, which relations must be recomputed and which errors can arise during processing. Chapter 2 “Representation of Spatial Objects” of the book Spatial Databases: With Application to GIS by Rigaux, Scholl and Voisard lays the foundation for these choices through entity and field models, geometric representation and the organization of object collections [1, Chapter 2, pp. 29–51].

Modern formats add ways of organizing data for different jobs: GeoJSON is convenient for exchanging features; TopoJSON represents shared arcs; KML ties data to geographic presentation; vector tiles deliver parts of a map by tile; GeoParquet organizes vector data in columns for analytical processing [15]–[19]. Comparing them by file size alone can mislead if the number of objects, level of detail, precision or spatial extent supplied differ.

The report poses three research questions. First, how do entity and field models relate to raster, vector, half-planes and topological structures? Second, which characteristics of modern formats inherit or change the way geometry is organized in the book chapter? Third, on the same data and reading task, how do boundary sharing, compression and tiling affect size, processing time and geometric consistency?

The scope centers on two-dimensional geometry comprising points, lines and polygons, with surface examples used to explain the field model. The experimental part uses controlled synthetic data, generated by source code and checked after conversion. The results allow observations about the measured data configurations and libraries; evaluating the overall performance of database systems, rendering time in browsers and real cadastral data lies outside this measurement.

The contributions of the report are a unified conceptual system from models to formats; original diagrams and pseudocode explaining the correctness conditions of representations; a comparison matrix of five formats; and an experimental suite with input data, configuration, raw measurements and reproduction code. The following sections present related literature, theory, technology updates, method, results and discussion.

2 Literature and Related Work

Chapter 2 “Representation of Spatial Objects” of Spatial Databases: With Application to GIS is the main organizing source [1, pp. 29–67]. Section 2.1 “Geographic Space Modeling” distinguishes entity and field models (pp. 31–34); Section 2.2 “Representation Modes” introduces tessellation, vector and half-planes (pp. 35–46); Section 2.3 “Representing the Geometry of a Collection of Objects” presents spaghetti, network and topology (pp. 46–51). Section 2.4 “Spatial Data Formats and Exchange Standards” has the TIGER/Line example (pp. 51–64); Section 2.5 contains the bibliographic notes (pp. 64–67). The chapter on spatial concepts in GIS: A Computing Perspective adds a computational view of geometry and topology [2]. Goodchild, Yuan and Cova provide a foundation for distinguishing geographic objects, fields and the ways of discretizing them; this work helps avoid equating a conceptual model with a specific storage structure [3].

The original papers were selected by the problem each clarifies. Egenhofer and Franzosa study topological relations through intersections of interiors and boundaries [4]. Günther and Wong develop a dual-space representation and the h-vector [5]. Realms raises the issue of a finite shared geometric domain and the consistency of operations [6]. Douglas and Peucker study reducing the points of a line [7]; De Floriani and Puppo study hierarchical triangulation for multiresolution surfaces [8]; Muller and Preparata use DCEL in the problem of intersecting convex polyhedra [9]. The report uses each source within exactly that scope, rather than attributing every feature of modern GIS to a single paper.

Table 1 Role of related experimental studies

Study	Object of evaluation	Use in the report
Wachs and Tzirita Zacharatou 2024 [10]	Loading Shapefile, GeoJSON, CSV and GeoParquet with different tool stacks.	Informs separating format from reading tool; adds checks of identical geometric output. The paper does not cover TopoJSON or KML.
Netek et al. 2020 [11]	Load metrics of raster tile and vector tile maps.	Informs separating initial load, zoom and pan; map load results are not used in place of whole-file read time.
Balla and Gede 2025 [12]	Rendering vector data in web mapping libraries.	Identifies the influence of libraries and the rendering process; the present experiments do not measure browsers.

Study	Object of evaluation	Use in the report
Li et al. 2017 [13]	Tiled vector model and representation of objects across tile boundaries.	Clarifies that an object can appear as several fragments across tiles. The model proposed in the paper is not the same as MVT.

Official specifications are used to establish data structures, coordinate systems, attribute semantics and scope of support [14]–[20]. Census documents are used to compare historical TIGER/Line with the current product family [21], [22]. Where a standard allows several configurations, the report states the one chosen in the experiments. Performance conclusions rest on the report's measurements, while specifications serve as the basis for the correctness of representations.

3 Theoretical Foundations of Spatial Representation

Entity model and field model

The entity model views geographic space through objects with identity. An object can be described by a triple of identifier, geometry and a set of attributes. For example, a land parcel has a parcel code, a boundary and land-use information; two parcels of the same shape are still two different objects if their identifiers or management semantics differ. Geometry does not replace object identity [1, Sec. 2.1.1, pp. 31–33], [3].

(1)

Here id is the identifier, g is the geometric component and A is the set of attributes describing the object.

The field model assigns a value to each location in the study domain. The value can be a measurement such as elevation or temperature, or a label such as soil type. Hence a field need not be a continuous function. In the ideal model the domain may be continuous; when stored on a computer, the field must be sampled or represented by a finite structure together with an interpolation rule [1, Sec. 2.1.2, p. 34], [3].

(2)

D is the spatial domain; V is the set of values; f(x,y) is the value of the attribute at location (x,y).

A phenomenon can be modeled in several ways depending on the question. Managing land-use rights requires parcel identifiers; describing soil type at each location may use a categorical field. This decision belongs to the conceptual model layer. Raster, vector and file formats belong to later layers. Entities can be encoded by raster cells carrying identifying labels; an elevation field can be represented by an irregular triangular mesh or contour lines [1, Sec. 2.2.1–2.2.2, pp. 35–42].

Raster and spatial discretization

A raster partitions the domain into cells, usually regular rectangles, and stores values by row and column. Recovering spatial meaning requires the grid origin, cell size, row and column counts and coordinate system. A cell can stand for the value of the whole cell area, or for a sample at its center or at a grid node. These two readings lead to different interpolation and area calculations, so they must be stated explicitly [1, pp. 35–37], [3].

When rasterizing a polygon, the rule that selects cells whose center lies in the polygon differs from the rule that selects every cell intersecting the polygon. The second rule can expand the selected area around the boundary; the first can miss objects narrower than the cell size. Reducing cell size improves the ability to describe detail but increases the number of cells. For a domain of fixed size, halving the cell side length increases the number of cells about fourfold in two dimensions.

Raster is convenient for neighborhood operations and map algebra because cell position is derived from indices. However, a grid does not by itself describe the identity and relations of real-world objects. Issues such as NoData values, resampling rules and the difference between continuous and categorical data must be handled according to the semantics of the data layer. This is why a raster–vector conversion is more than changing a file extension.

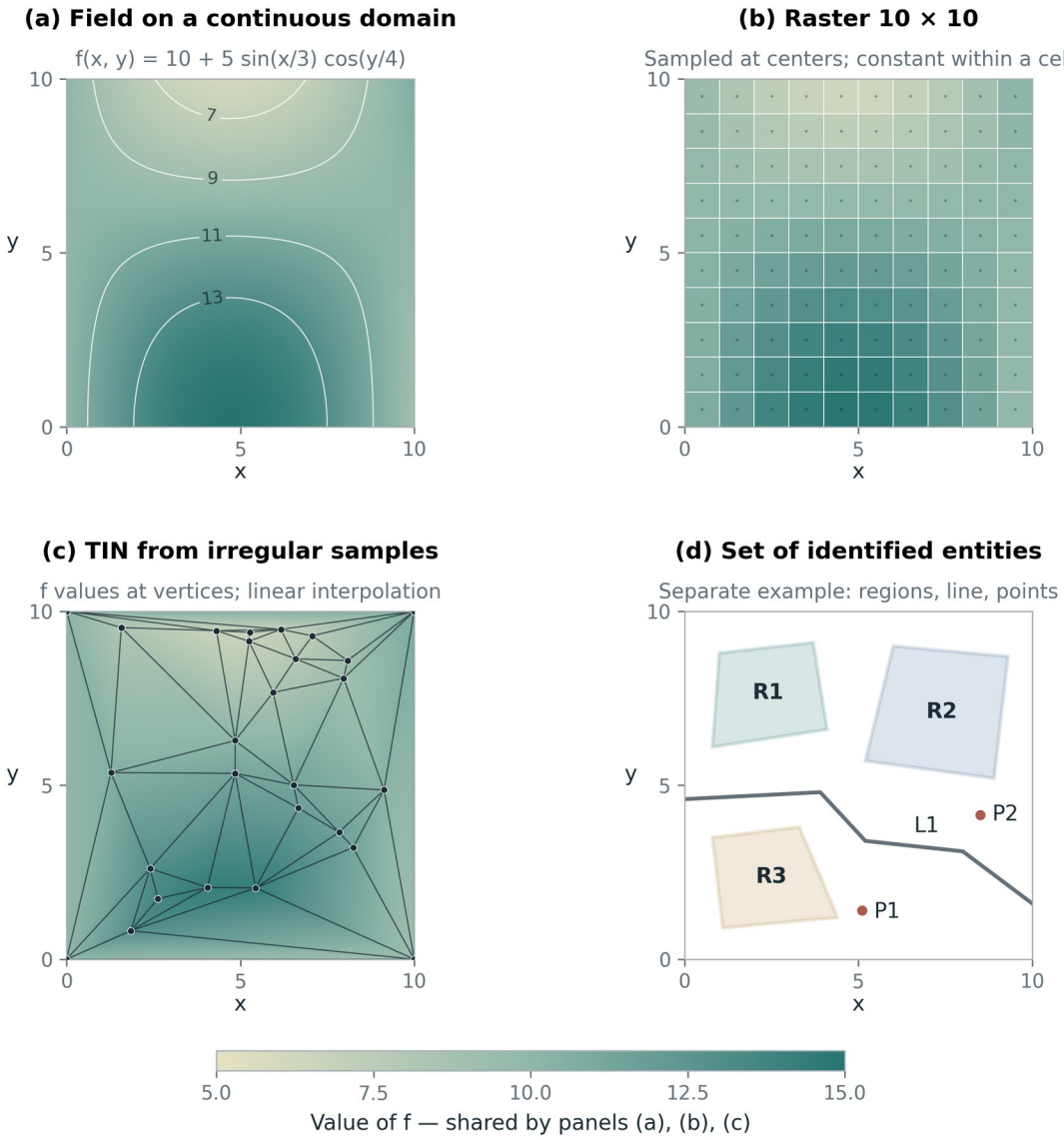


Figure 1 Panels a–c represent the same synthetic field through a function, a raster and a linearly interpolated TIN; panel d illustrates a separate set of entities. Original figure with illustrative data; conceptual frame based on [1, Sec. 2.1–2.2, pp. 31–42], [2], [3].

Vector representation and surfaces with TIN

Vector represents geometry with elements such as points, polylines and polygons. A point stores a location; a polyline stores an ordered sequence of vertices; a polygon has an outer boundary and may have hole rings. A coordinate list alone does not guarantee a valid polygon: closure, self-intersection, ring structure and how rings relate must be checked. Simple Feature Access provides the geometry framework and related conditions for these object types [1, pp. 38–42], [14].

In a polyline, an intermediate vertex may serve only to describe shape; it is not necessarily a network node. Similarly, a MultiPolygon object can comprise several disjoint areas yet have a single identifier. The distinction among geometric component, data object and real-world object must be kept when counting objects or comparing conversion results.

A TIN is a partition into triangles that may differ in size and shape. Elevation or another quantity is assigned to the vertices; a rule, usually linear interpolation over the triangle, determines values inside. TIN is an example showing that a field can have a vector representation. Hierarchical TINs add a multi-level organization of detail, but not every TIN is hierarchical or a Delaunay triangulation [1, Sec. 2.2.2, pp. 38–42], [8].

Reducing the vertex count is one way to reduce vector size, but it changes the geometry. The Douglas–Peucker algorithm selects the point farthest from the segment joining the two ends, then keeps subdividing if the deviation exceeds a threshold [7]. When applied independently to adjacent polygons, the same boundary may be simplified differently. Therefore smaller size must be viewed together with geometric error and the consistency of the object collection.

Half-plane representation

A closed half-plane is defined by a linear inequality with a nonzero normal. The intersection of finitely many half-planes is a convex set; if this set is bounded, non-empty and has an interior, it can represent a convex polygon. The edges are determined by the corresponding boundary lines. This is a representation by the constraints of a region, complementing the listing of vertices and edges [1, Sec. 2.2.3, pp. 42–46].

(3)

m is the number of constraints; (a_i,b_i) is the normal of the boundary line; c_i determines the position of the boundary line.

A nonconvex polygon generally requires a union of convex components or a suitable Boolean expression. Not every nonconvex polygon can be described by an intersection of half-planes alone. The dual-space and h-vector method of Günther and Wong further organizes polyhedral chains and set operations; the point test algorithm below is a limited illustration, not a complete implementation of that method [5].

(4)

$p=(x_p,y_p)$ is the point to classify; d_i is the signed distance to the boundary line of constraint i in the planar coordinate system in use.

Algorithm 1 assumes that the constraints describe a non-empty convex polygon with an interior. The threshold ε has the same unit as the coordinates and is non-negative. Normalizing the normal ensures

that multiplying an inequality by a positive constant does not change the meaning of the tolerance. If ε is chosen in meters, a suitable coordinate system must be used; degrees of longitude and latitude cannot be treated as meters.

Algorithm 1 Classifying a point with half-planes

```
CONVEX_CLASSIFY(p, constraints, epsilon)
    near_boundary <- false
    for each (a, b, c) in constraints
        require hypot(a, b) > 0
        d <- (a*p.x + b*p.y + c) / hypot(a, b)
        if d > epsilon: return OUTSIDE
        if abs(d) <= epsilon: near_boundary <- true
    if near_boundary: return BOUNDARY
    return INSIDE
```

The algorithm needs O(m) time and O(1) auxiliary memory; the norms of the normals can be precomputed. With ε equal to 0 and exact arithmetic, the result is the mathematical set classification. With positive ε, BOUNDARY denotes a tolerance band and may include points slightly outside the region. This tolerance is not a proof of robustness for all floating-point computations.

For example, the triangle with vertices (0,0), (2,0), (0,2) is defined by the conditions that x is non-negative, y is non-negative and x+y does not exceed 2. With ε equal to 0, the point (0.5, 0.5) is inside, the point (1, 1) is on the boundary and the point (1.5, 1) is outside. For an L-shape made of two rectangles, an OR can test whether a point belongs to the union; however, BOUNDARY labels cannot be ORed to infer the global boundary. The point (0.5, 1) on the edge dividing the two pieces can lie inside the union.

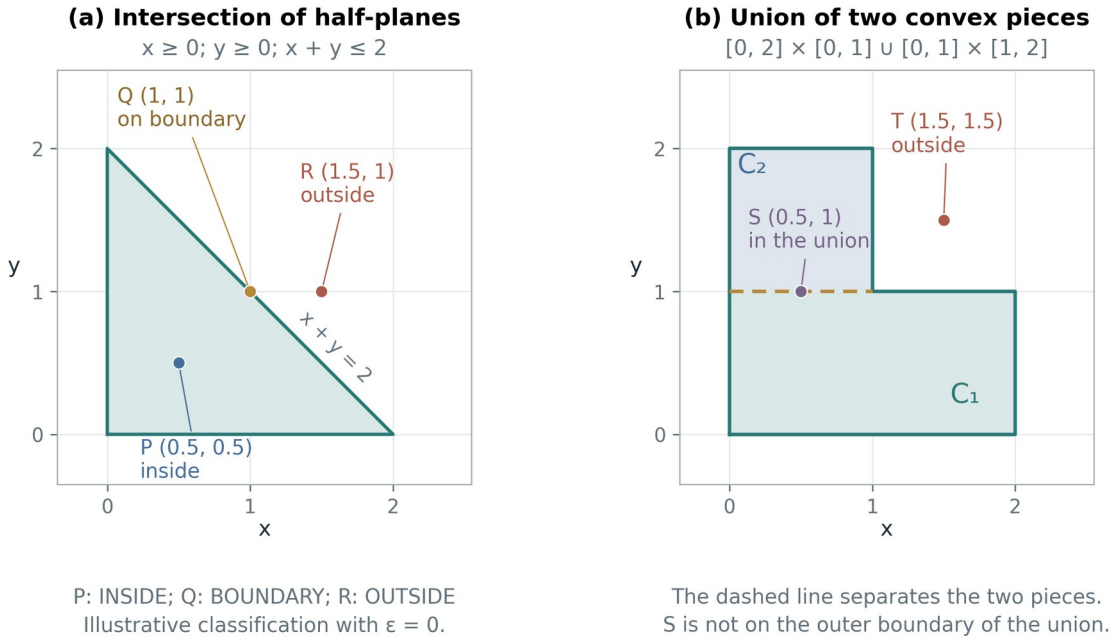


Figure 2 Original example of half-planes and unions of convex regions. An edge dividing the pieces is not necessarily a boundary of the union. Representation basis from [1, Sec. 2.2.3, pp. 42–46], [5].

Organizing object collections with spaghetti, network and topology

In the spaghetti model, each object keeps its own geometry. When two polygons are adjacent, the coordinates of the shared boundary may be stored twice. This organization makes individual objects easy to exchange, but adjacency usually has to be inferred or stored additionally. If the two copies of a boundary are edited differently, the dataset can develop gaps or overlaps although each polygon remains individually valid [1, Sec. 2.3.1, pp. 47–48].

The network model uses nodes and arcs to represent connectivity. Arcs can carry direction, weights and traffic attributes. Two roads crossing in projection is not enough to conclude that they connect, as with an overpass. Building a network therefore requires rules about connection semantics, not merely an operation that finds segment intersections [1, Sec. 2.3.2, pp. 47–49].

For a planar partition, the topological model can store nodes, arcs and faces, where a directed arc links to its left and right faces. Two adjacent regions share one arc; updating the arc can change both regions at once in a representationally consistent way. This benefit depends on the input data and the constraints maintained, and does not by itself guarantee that the data match reality [1, Sec. 2.3.3, pp. 49–51].

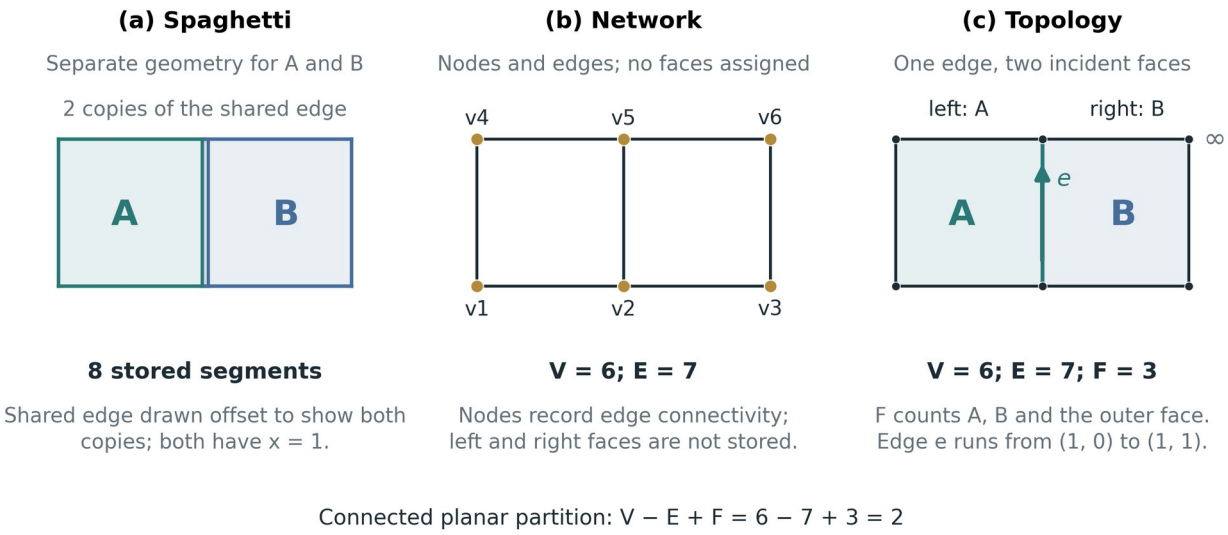


Figure 3 The same two-square example organized as spaghetti, network and topology. Original figure; separating the two boundary copies in the spaghetti panel only illustrates storage. Concepts follow [1, Sec. 2.3, pp. 46–51].

Table 2 Comparison of three ways of organizing an object collection

Aspect	Spaghetti	Network	Topology of a planar partition
Main elements	Separate geometry of each object	Connecting nodes and arcs	Nodes, arcs and faces
Relation made explicit	Identity and geometry of each object	Connectivity and direction	Incidence and adjacency among faces

Aspect	Spaghetti	Network	Topology of a planar partition
Shared boundary	Coordinates may be repeated	Faces on the two sides not defined by default	One arc and the faces on both sides can be stored
Risk to control	Inconsistent boundary copies	Geometric intersection misread as connection	Noding, ring orientation and incidence consistency

A stored topological structure must be distinguished from a computed topological relation. Egenhofer and Franzosa describe relations by the emptiness or non-emptiness of intersections of interiors and boundaries [4]. Simple Feature Access specifies a DE-9IM relation framework with interior, boundary and exterior [14, Sec. 6.1.15]. Two objects touching at a point differ from two objects sharing a boundary segment. In the experiments, adjacency is defined specifically as sharing a boundary of positive length.

DCEL in the paper by Muller and Preparata organizes information through the fields V1, V2 for the two endpoints, F1, F2 for the two faces adjacent to the edge, and P1, P2 for links to the next edge around the two vertices [9, Sec. 2]. The explanation using two oppositely directed half-edges with origin, twin, next, prev and face links is a common implementation variant for understanding traversal. The field list of this variant should not be taken as a verbatim description of DCEL in the 1978 paper. Realms further raises the issue of a finite and consistent shared geometric domain; merely rounding coordinates is not equivalent to building a Realm [6].

Building a shared-edge table and its correctness conditions

Algorithm 2 illustrates merging boundary segments in an already noded partition. Noding is the step of splitting segments at the necessary intersections or connection points. The input must consist of valid polygons with non-overlapping interiors; the two sides of a shared boundary have the same coordinates and the same segmentation. Outer rings are oriented counterclockwise and hole rings clockwise, so that the polygon interior lies on the left when moving along a ring.

Algorithm 2 Building a shared-edge table from noded segments

```
BUILD_SHARED_EDGES(polygons)
    E <- empty dictionary
    for each face f in polygons
        for each ring r of f
            for each oriented segment u -> v of r
                reject zero-length segment
                key <- (min_lex(u,v), max_lex(u,v))
                if key not in E: E[key] <- empty edge
                side <- LEFT if (u,v)==key else RIGHT
                if E[key][side] assigned: reject conflict
                E[key][side] <- f
                append oriented reference to ring r
    return E and references for each ring
```

The canonical key allows the same segment to be recognized when two rings traverse it in opposite directions. With S segment occurrences, a hash table gives expected O(S) time and O(S) memory; a balanced tree gives O(S log S) time. The costs of intersection finding, noding, snapping and validity checking are outside these estimates. The result is an edge–face table with ring references, not yet a

complete DCEL builder; boundary components, traversal links and the outer face still have to be organized.

In the example of two adjacent squares in Figure 3, the input has eight segment occurrences but only seven unique edges and six vertices. The directed edge from (1,0) to (1,1) has left face A and right face B. Counting the outer region, this connected planar graph satisfies $V-E+F=6-7+3=2$. If the left boundary of B alone is moved to $x=1+\delta$, a gap of area δ appears with height 1; merging by coordinates does not by itself fix the gap.

TIGER Line as an example integrating geometry and topology

The TIGER/Line example in Section 2.4.2 of Chapter 2 of the book uses the 1998 dataset. A complete chain combines the shape of a line with two end nodes and the polygons on its two sides; GT-polygons are formed from chains. In the book's description, record 1 holds the TLID and end coordinates, record 2 holds the shape points, and record I links a chain to its left and right polygons [1, Sec. 2.4.2, pp. 54–61]. This structure illustrates how objects are recovered from shared components and linking keys.

The MAF/TIGER database must be distinguished from the Census distribution products. The 2026 TIGER/Line documentation has separate parts for shapefiles and relationship files [21]. The Census product guide distinguishes products by format and intended use [22]. Hence the records of the 1998 example are used to explain the principle; the report does not apply that historical file structure unchanged to current shapefiles.

4 Modern Formats and Technologies

Exchanging features, maintaining shared boundaries, presenting on a globe, loading maps and analyzing tables require different ways of organizing data. Table 3 compares structure and purpose; the subsections clarify the information to control during conversion [15]–[19].

Table 3 Structure and scope of use of the five formats

Format	Encoding and organization	Relations among objects	Suitable work
GeoJSON [15]	JSON text; Feature and FeatureCollection.	Geometry stored separately; adjacent boundaries may repeat coordinates.	Exchanging features and API integration.
TopoJSON [16]	JSON; geometry references arcs.	Arcs can be shared in both directions.	Exchanging regions with many shared boundaries.
KML [17]	XML; objects, presentation styles and extended data.	No shared-arc structure among Placemarks.	Geographic presentation, annotation and globes.
MVT [18]	Protocol Buffers; layers and features in each tile.	Separate geometry; an object can be split across tiles.	Delivering maps by viewport and zoom level.
GeoParquet [19]	Columnar Parquet; geometry and attribute columns.	Boundaries encoded separately in each geometry.	Table analysis and exchange of large vector data.

GeoJSON feature exchange with a defined coordinate system

RFC 7946 specifies GeoJSON coordinates in longitude, latitude order, in decimal degrees on WGS84, equivalent to OGC:CRS84. A third component, if present, is height in meters above the WGS84 ellipsoid. Therefore data in a projected planar system must be suitably transformed before being exported as conforming GeoJSON [15, Sec. 3.1.1 and 4].

A Feature accepts a null geometry, and properties is a JSON object or null. Reading and writing must preserve these cases correctly. The number of decimal digits affects size but does not express measurement uncertainty; bbox describes extent and does not itself create an index [15, Sec. 3.2, 3.1.10 and 5]. Repeated boundary coordinates should be considered when exchanging many adjacent regions.

TopoJSON arc sharing and quantization control

TopoJSON connects directly to the idea of storing shared boundaries in Section 3. Lines and polygon rings reference arcs in the arcs array. A non-negative reference uses an arc in its stored direction; a negative reference j uses the arc with index ~j = −j−1 in the reverse direction. For instance, −1 denotes arc 0 reversed, not arc 1 [16, Sec. 2.1.4].

Transform is an optional member. With quantization, the first two coordinate components are mapped to an integer grid and arcs are delta-encoded. This can reduce size but changes precision. To evaluate the effect of arc sharing alone, the baseline configuration must disable quantization and simplification. The CRS must be agreed by the exchanging parties. Arc references do not by themselves provide traffic rules, weights or a complete adjacency table [16, Sec. 2.1–2.2].

KML geometry tied to presentation

KML suits cases where objects need names, descriptions, icons, line styles or a way of displaying height. Coordinates take the form longitude,latitude[,altitude]. The meaning of altitude depends on altitudeMode; the vertical reference of the standard uses the EGM96 geoid, unlike the ellipsoidal height in GeoJSON. Converting 3D data therefore requires checking both the height datum and the display mode [17, Sec. 9.20, 16.10 and Annex A].

Schema and SimpleField declare attribute types; SchemaData and SimpleData supply the corresponding values. The experiment must keep the numeric type of object codes and avoid reading all XML values as strings. KML 2.3 continues to use the namespace http://www.opengis.net/kml/2.2. Text KML and compressed KMZ must be distinguished when comparing size [17, Sec. 9.5–9.6, 9.11 and Annex C]. The report uses 2D geometry, so it does not fully evaluate the 3D presentation abilities of KML.

Vector tiles delivering data by viewport

MVT 2.1 encodes layers and features with Protocol Buffers. Geometry uses integer coordinates in the tile's local system; the reader must learn the position, projection and tiling scheme externally. Web Mercator, XYZ and extent 4096 are configuration choices, not mandated by the standard [18, Sec. 3–4]. Attributes use scalar values; JSON null has no direct encoding type [18, Sec. 4.1].

Quantization can shift vertices or lose small details. Clipping at tile boundaries can split an object into several fragments; buffer and simplification are choices that must be published. Therefore object codes, duplicate fragments and post-decoding error must be checked, and the size of a viewport must be separated from the size of the whole dataset [18]. OGC API Tiles specifies access to tile and tileset resources; MVT specifies the encoded content. The two layers complement each other [20, Sec. 7–9 and 16.7].

GeoParquet columnar tables and data filtering conditions

GeoParquet adds geographic metadata to Parquet. Version 1.1.0 is chosen as the experimental baseline, with the geometry column encoded in WKB. The CRS can be declared in PROJJSON: a missing crs field means CRS84, while a null crs means the CRS is unknown or undefined. The two cases must not be treated alike [19].

Columnar organization lets a reader select the columns it needs. The file-level bbox and per-row covering.bbox can help skip irrelevant data; the effect also depends on spatial ordering, row groups, statistics and the reader. They do not guarantee a global spatial index. A compression codec such as ZSTD must also be stated, because size and time results are influenced by both structure and compression. Initialization costs can be significant for small files; the specification does not guarantee that GeoParquet always reads faster [19].

From specifications to experimental design

E1 tests shared boundaries and consistency; E2 measures the cost of storing and reading the complete dataset, with checks on preservation of geometry and attributes; E3 measures tiled data by viewport along with quantization error. This separation of tasks ties each conclusion to the amount of information supplied and the work actually measured.

5 Experimental Method

Three experiments were designed to connect the concepts of Chapter 2 “Representation of Spatial Objects” [1, pp. 29–67] with the behavior of modern formats. E1 examines shared boundaries and consistency during updates. E2 compares the size and processing time of a complete object collection whose geometry is preserved. E3 examines tile-based delivery and the quantization error of MVT. Separating the three problems avoids comparing a complete dataset with a set of display fragments and then treating them as equivalent products.

These are actual runs on synthetic data, in the computing environment used to prepare the report, on November 2, 2026. Data, code, configuration, logs and per-repetition results are stored in the reproduction package. The figures are not field measurements, are not taken from the cited papers, and are not yet considered results the group ran on its own machine. The method draws on how [10] identifies processing pipelines and how [11], [12] separate data loading from display; the design here has its own scale and scope.

Data and runtime environment

The data consist of two-dimensional polygon partitions in the longitude range 105.75–105.87 and latitude range 20.95–21.07, in OGC:CRS84. The three scales are 100 objects on a 10 × 10 grid, 1,000

objects on a 25 × 40 grid and 10,000 objects on a 100 × 100 grid. Each ring has 17 coordinate pairs, including the closing point. An object has two integer attributes: a unique id, and class, the remainder of the sum of the row and column indices divided by 5. Geometry is created before the timed interval.

Each interior grid edge has four segments and a zigzag offset of 6% of the cell size; two adjacent polygons use exactly the same coordinate sequence, reversed when necessary. This design preserves shared boundaries and reduces the risk of confusing arc sharing with the removal of collinear vertices. The outer boundary remains straight, so the effect of removing collinear vertices is not fully eliminated. Coordinates are rounded to 12 decimal places to make the data deterministic; this figure does not describe real-world survey precision.

The run used Python 3.12.14 on Linux x86_64. The host reports an AMD EPYC 9V74 CPU, but the environment was granted only a quota equivalent to 8 CPUs and a memory limit of 8 GiB; this is not to be read as a test using all 80 cores named by the CPU. The main libraries are Shapely 2.2.0, GEOS 3.14.1, GeoPandas 1.2.0, PyArrow 26.0.0, topojson 2.2, mapbox-vector-tile 2.2.0, pyproj 3.8.0 and PROJ 9.8.1. The file environment.json records all versions and the start and end times in UTC.

E1 Shared boundaries, gaps and updates

Experiment E1 uses two adjacent polygons in four scenarios. First, the two boundaries match exactly. Second, polygon B is shifted by 10^{-6} degrees of longitude in the direction that creates a gap. Third, a single mid-boundary vertex of A alone is moved by a further 10^{-4} degrees in GeoJSON. Fourth, the same position is moved on the arc referenced by both A and B in TopoJSON. The two endpoints connecting to other arcs stay fixed. TopoJSON quantization and simplification are both disabled.

The TopoJSON library may remove collinear vertices in the two rectangles. Therefore, before the edit, the code reinserts the intermediate vertices on the arc so that both update methods act on the same position. The touch relation is checked on the original geometry; illustrative distance, length and area are computed after projecting to UTM 48N, EPSG:32648. The metrics are the number of shared arcs, validity, touching, distance and overlap area. The test covers only this specific operation and does not assume that every arc edit is valid.

E2 Size and processing of the complete dataset

Table 4 Encoding configurations in experiment E2

Configuration	Encoding and attributes	Control settings
GeoJSON	Compact UTF-8 FeatureCollection; numeric id and class	Round-trip floating-point coordinates; no simplification
TopoJSON	Shared arcs; id and class attributes	prequantize, topoquantize, presimplify, toposimplify disabled
KML	Polygon; Schema and int SimpleField for both attributes	No altitude; clampToGround; round-trip floating-point
GeoParquet NONE	WKB; schema 1.1.0; no column compression codec	No index written; row group 1,000
GeoParquet ZSTD	Same data and schema as NONE; ZSTD codec	Only the codec setting changes; geometry unchanged

The configurations are based on specifications [15]–[19]. Raw files are measured in bytes. A separate size measurement compresses each whole file with gzip level 6 and mtime set to 0. External gzip and ZSTD inside Parquet are two different mechanisms: the GeoParquet ZSTD file in the “file” column is already compressed internally; its gzip column is an additional compression reported for transparency, not a deployment recommendation.

The read path of every configuration ends in the same form of result: a complete list of records holding id as int, class as int, and a constructed Shapely geometry. Thus no configuration reads only metadata while another must build all geometry. However, the readers differ: KML uses lxml, GeoParquet uses GeoPandas/PyArrow, and TopoJSON uses the Polygon decoder shipped with the experiment code. The measured times reflect these specific pipelines, including library costs.

Each configuration has one correctness-check pass and one untimed warm-up pass, followed by five measurements. The order of configurations is shuffled within each repetition using seed 20261102. The clock is time.perf_counter_ns; garbage collection is invoked outside the timed interval. Write time covers building the target structure, encoding, writing and closing the file, but no fsync, so it does not represent durable write latency to the device. Read time covers opening the file, reading, decoding and creating geometry.

The operating system cache is warm and is not cleared; each read follows a write of the same configuration. The report uses the median and Q1–Q3 range of the five runs as descriptive statistics and performs no significance testing. E2 produces 75 measurement rows, corresponding to three scales × five configurations × five repetitions; each row holds both write and read times. The costs of generating data and checking results are outside the timed interval.

Post-read checks cover the object count, unique and matching ids, correct attribute values and integer types, valid and non-empty geometry, topological equality, maximum Hausdorff distance and area deviation. Hausdorff uses the discrete GEOS/Shapely measure without added densification. The check threshold is set at 10^{-12} degrees; the report presents the measured deviation rather than substituting the threshold for it. “Lossless” in E2 means only that the geometric point set and the checked attributes are preserved; it does not require preserving vertex order or the number of collinear vertices.

E3 Vector Tiles by zoom level and viewport

E3 uses 10,000 objects, projected to EPSG:3857, with the XYZ scheme, extent 4096 and buffer 0, at zoom levels 11, 13 and 15. This is the configuration of the test, not a set of coordinate systems or parameters mandated by MVT [18]. No line simplification is applied. Polygons are clipped at tile boundaries; only Polygon or MultiPolygon fragments with positive area are kept, then quantized to the nearest integer grid and encoded with mapbox-vector-tile.

The viewport lies at the center of the domain, one quarter of the domain width and one quarter of its height, so its area is one sixteenth. A tile is selected when it intersects the viewport; all objects in a selected tile are read. Therefore the number of ids received can exceed the number of objects that actually intersect the viewport. The report separates the number of tiles, the number of geometry fragments and the number of unique ids. The gzip total is obtained by compressing each tile and summing the bytes, not by compressing the whole directory as one stream.

After decoding, coordinates are mapped through a global integer grid before conversion to floating point, so that the same node in two adjacent tiles has the same numeric value. Fragments with the same id are merged to check geometry, attributes and error against the source. Hausdorff error is measured in meters in the Mercator coordinate system; it is not to be interpreted directly as distance on the ground. The area difference ratio is defined per object as follows.

(5)

P_i is the source polygon and $\hat{P}_i$ the polygon after merging fragments; $\triangle$ is the symmetric difference. Area is computed in EPSG:3857. A region reassigned between two objects may contribute to the numerator of both; this is not the difference between the outlines of the union of the whole map.

Decoding time has one warm-up and five measured repetitions, covering local file reads, Protocol Buffers decoding, Shapely geometry creation and coordinate conversion. Tile build time is measured only once as an operational log entry and is not used as stable comparative evidence. E3 does not measure the network, HTTP, browsers, map drawing or user interaction. Because of clipping and quantization, MVT is evaluated separately rather than placed in the E2 tables as an equivalent complete copy.

6 Experimental Results

E1 Boundary sharing supports updates but does not repair data

Table 5 Results of the four shared-boundary scenarios

Scenario	Shared arcs	Relation after processing	Main quantity
Exactly matching boundary	1 of 3 arcs	The two polygons touch; both valid	Distance 0 m; overlap 0 m²
10^{-6} degree gap	0 of 2 arcs	Not touching; both valid	Gap of 0.103930 m remains
Edit A alone in GeoJSON	Stored separately	Overlapping; each polygon valid	Overlap area 2,875.871 m²
Edit shared arc in TopoJSON	1 of 3 arcs	Still touching; both valid	Overlap area 0 m²

With matching input boundaries, TopoJSON creates one shared arc referenced by both polygons; the touch relation holds after reading back. With an input gap of about 0.104 m, converting to TopoJSON neither creates a shared arc nor removes the gap. This result refutes the reading that the name “topology” by itself guarantees correct data: the storage structure and the process of checking and repairing are separate tasks.

The independent edit creates an overlap although both polygons remain individually valid. Therefore an is_valid check per object is not enough to guarantee a non-overlapping partition. The shared-arc edit keeps the two boundaries coincident, with no overlap in this case. The observed benefit comes from both objects referencing the same boundary data; checks for self-intersection, ring inversion or collision with other edges are still needed after more complex edits.

E2 Size and speed depend on configuration and scale

All 15 combinations of scale and configuration retain every object, id and class; all geometry read back is valid, non-empty and topologically equal to the source. The largest measured Hausdorff distance is 0 degrees. The largest relative area deviation of TopoJSON is approximately 2.26×10^{-16}, consistent with the effect of floating-point arithmetic; the other configurations have a value of 0 in this measurement set. This is the condition for comparing performance on the same geometric information.

Table 6 Size and median time with 10,000 polygons

Configuration	File bytes	Gzip6 bytes	Write ms	Read ms
GeoJSON	4,551,029	538,339	321.825	323.879
TopoJSON	3,017,111	439,887	5,313.076	371.297
KML	6,511,224	546,117	503.613	525.340
GeoParquet NONE	3,023,469	536,556	30.605	55.123
GeoParquet ZSTD	452,131	253,110	32.108	50.761

Tables 5–10 and Figures 4–5 come from the run accompanying the report. The “file” column of GeoParquet ZSTD already includes column compression. The values in Table 6 are medians of five measurements, not the time of one repetition chosen for illustration.

Table 7 Q1 to Q3 range of times at 10,000 polygons

Configuration	Write Q1–Q3 ms	Read Q1–Q3 ms
GeoJSON	311.682–387.404	298.862–326.171
TopoJSON	5,086.194–6,410.771	368.037–425.634
KML	489.318–519.279	480.217–597.012
GeoParquet NONE	26.229–32.296	54.856–65.988
GeoParquet ZSTD	31.720–39.108	49.235–58.349

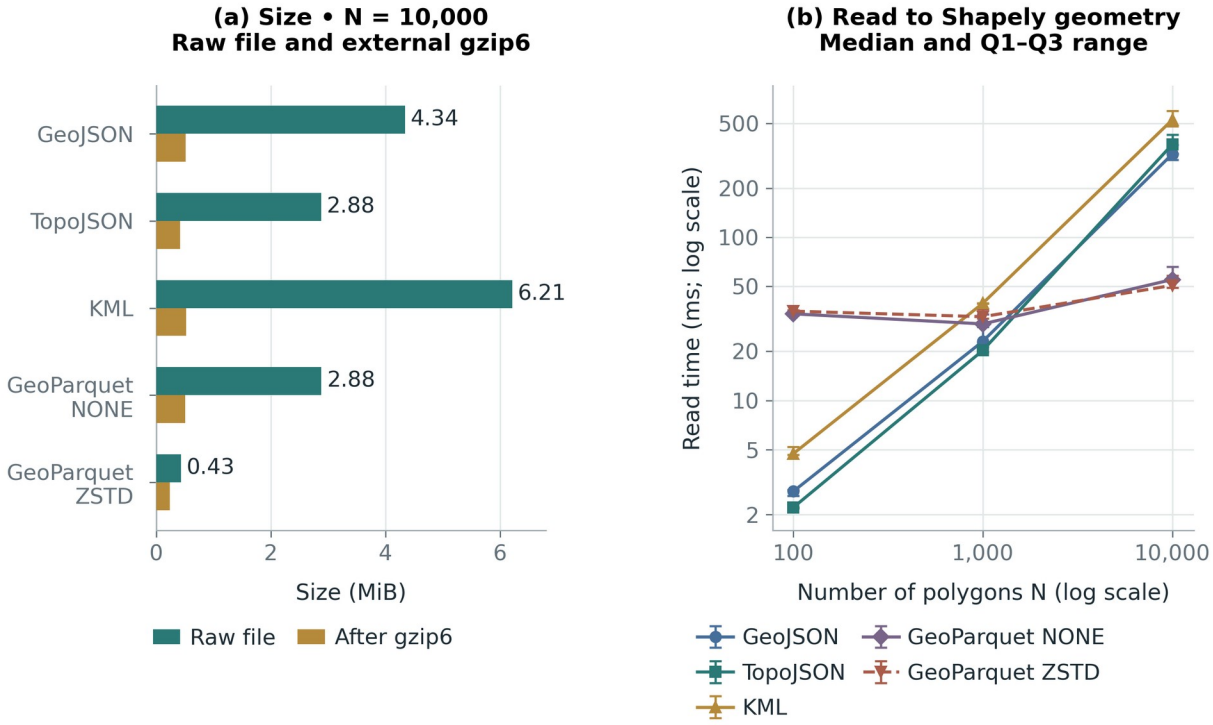
At 10,000 objects, TopoJSON reduces file size by 33.70% and gzipped size by 18.29% compared with GeoJSON. The structure has 20,196 arcs, of which 19,800 are shared, with 99,796 coordinate pairs in the arcs; GeoJSON holds 170,000 ring coordinate pairs. The byte reduction does not equal the reduction in coordinates, because of arc indexes, JSON structure and attributes. Some collinear vertices on the outer boundary may be removed without changing the geometry; hence not all of the saving is attributed to arc sharing.

Smaller size does not come with shorter write time. Building and writing TopoJSON takes a median of 5,313.076 ms, against 321.825 ms for GeoJSON in this pipeline. The raw KML file is larger than GeoJSON, but after gzip the two sizes are close: 546,117 and 538,339 bytes. Looking only at the raw file overlooks how well repeated text structures compress.

Table 8 Median read time by data scale

Configuration	100 objects ms	1,000 objects ms	10,000 objects ms
GeoJSON	2.774	23.058	323.879
TopoJSON	2.211	20.280	371.297
KML	4.723	39.441	525.340
GeoParquet NONE	33.960	29.460	55.123
GeoParquet ZSTD	35.265	32.624	50.761

GeoParquet ZSTD reads 10,000 objects in 50.761 ms, below the 323.879 ms of GeoJSON in the measured environment. However, with 100 objects, GeoJSON needs only 2.774 ms while GeoParquet ZSTD needs 35.265 ms. The reversal is consistent with the costs of opening, interpreting metadata and creating intermediate structures possibly dominating small datasets. The 1,000-object result further shows that an exact crossover threshold should not be inferred from only three scale points. Further measurement is meaningful only once the data type and pipeline of a specific application are defined.



GeoParquet ZSTD is already compressed internally; gzip6 measures an extra outer layer.

5 repetitions after warm-up, warm cache; the read result includes id, class and geometry.

Figure 4 Size at 10,000 objects and read time by scale. The time chart uses the median and Q1–Q3 range of five repetitions; log scales are marked on the axes. Source: experiment E2.

E3 Small tiles reduce over-fetching but increase the full-domain tile count

Table 9 MVT tile counts and data volume by viewport

Zoom	Full-domain and viewport tiles	Full-domain gzip bytes	Viewport gzip bytes	IDs received in viewport tiles
11	4 / 2	128,913	114,625	9,100
13	16 / 4	162,451	77,859	5,175
15	156 / 16	311,855	35,424	1,295

The viewport actually intersects only 676 source objects. All three zoom levels retrieve these 676 ids, but the number of ids in the tiles to be read is 9,100, 5,175 and 1,295 respectively. The reason is that a tile can extend far beyond the viewport. Hence a viewport covering one sixteenth of the area does not mean reading only one sixteenth of the bytes. The number of fragments in the viewport tiles is larger still than the number of ids because an object may cross a tile boundary: 9,191, 5,320 and 1,520 fragments respectively.

Going from zoom 11 to 15, the full-domain tile count rises from 4 to 156 and the full-domain gzip total rises from 128,913 to 311,855 bytes. Meanwhile, the gzip needed for the viewport falls from 114,625 to 35,424 bytes. These two quantities reflect two different goals: storing or reading the whole map, and retrieving only the area being viewed. For a specific viewport, many small tiles can reduce over-fetching, but this alone says nothing about connection costs or interaction latency.

Table 10 Full-domain geometric error and viewport decoding time of MVT

Zoom	Grid step m Mercator	Max Hausdorff m Mercator	Area difference %	Median decoding ms
11	4.777314	3.371148	2.711787	678.333
13	1.194329	0.837874	0.699537	387.395
15	0.298582	0.209441	0.174150	87.691

All 10,000 ids are recovered from the full-domain tiles, attributes match and the merged polygons are valid. Even so, the geometry has changed through quantization. Discrete Hausdorff falls from 3.371148 m Mercator at zoom 11 to 0.209441 m at zoom 15; the area difference ratio of Equation 5 falls from 2.711787% to 0.174150%. This illustrates that validity and id preservation are not the same as keeping the geometry unchanged.

The viewport decoding time has a Q1–Q3 range of 659.707–751.373 ms at zoom 11; 386.291–390.163 ms at zoom 13; and 85.931–89.362 ms at zoom 15. Fewer fragments to read is consistent with lower decoding time. These times cannot be used to conclude directly about drawing speed or the advantage of a web library; those quantities require a separate measurement design as in [11], [12].

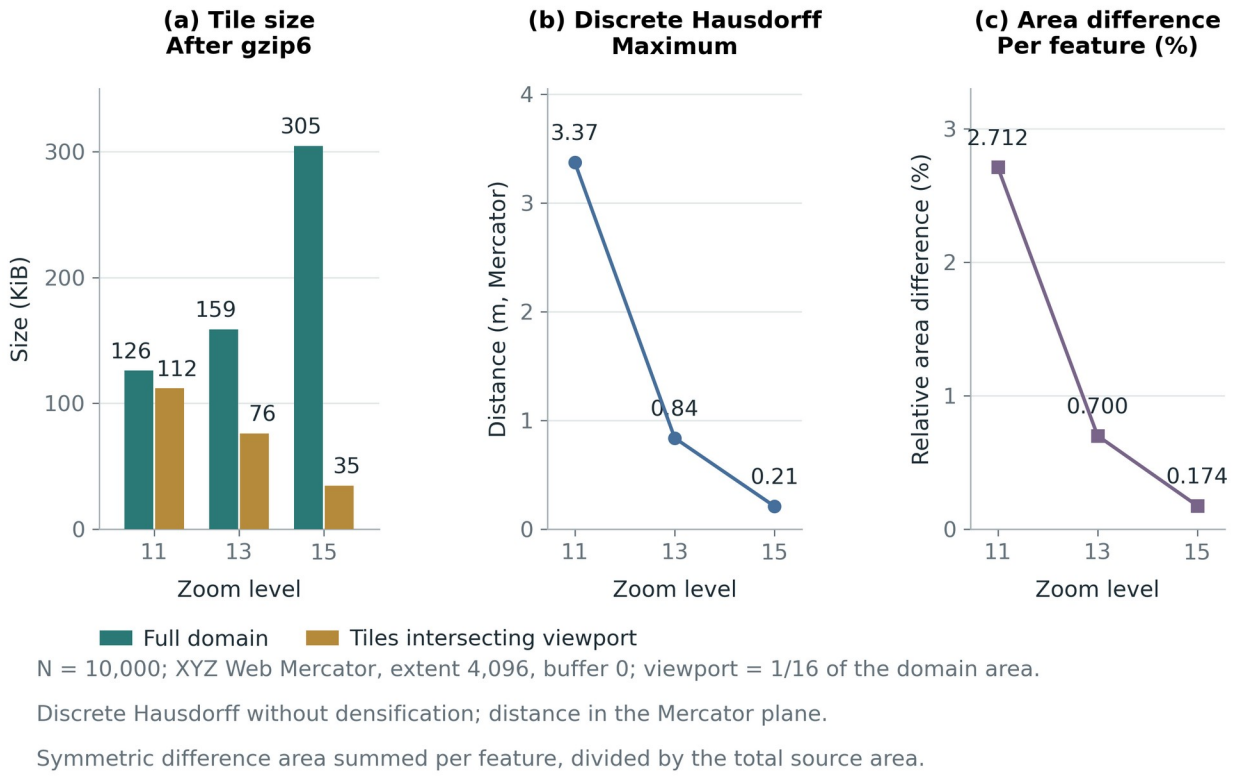


Figure 5 MVT shows the trade-off between size by read domain and quantization error. The error is a discrete measure in the Mercator system; the area ratio follows Equation 5. Source: experiment E3.

7 Discussion and Limitations

From model to usage decision

The first question is answered by distinguishing three layers. The entity or field model determines what counts as the object of study. Raster, vector, TIN or half-planes determine how geometry or spatial values are described. Spaghetti, network and topology determine how relations among components are organized. Formats then specify how these are encoded and exchanged. Using vector coordinates in common does not make GeoJSON, TopoJSON, KML, MVT and GeoParquet serve the same data contract [1, Sec. 2.1–2.4, pp. 31–64], [2], [3], [15]–[19].

The second question must be assessed by the invariants to be kept. When managing boundaries, the invariant may be that a shared edge has a single update source and that the partition has no gaps or overlaps. When exchanging records, the invariants are id, attributes, geometry type and CRS. When displaying by tile, the contract may accept clipping, reduced detail and quantization at a suitable error level. The conditions to keep must be stated before conversion and exactly those conditions checked afterward, rather than only checking that the file opens.

Table 11 Suggested choices by task and conditions to check

Task	Option worth considering	Conditions to check
Exchanging objects through services or processing small sets	GeoJSON	Longitude-latitude order, null values, digit count, attribute preservation
Sharing the boundaries of a partition	TopoJSON or a database with topology	Noding, coincident boundaries, shared arcs, validity after edits
Exchanging data with presentation on a map or globe	KML or KMZ	Attribute types, altitudeMode, height system and software support
Displaying maps by viewport and at many zoom levels	Vector Tiles with tiling rules and a delivery service	CRS, extent, buffer, geometry loss, cross-tile ids and over-fetching
Batch analysis of geographic tables or access to some columns	GeoParquet	Schema version, CRS, codec, row group, metadata and reader capability

Table 11 is a conditional interpretation of the specifications and experiments, not a ranking of formats. A system may store an analytical dataset in GeoParquet, export GeoJSON for exchange, build MVT for display and maintain topology in the source database. In that case, consistency among the products depends on the data generation process, source versions and post-conversion checks; it is not guaranteed merely because all the products carry coordinates [14]–[20].

Critique of easily mistaken interpretations

A file with coordinate fields does not prove it is a geometric data source able to replace the original. MVT shows this clearly: all ids and attributes can remain intact while the boundaries have been quantized. Conversely, TopoJSON in E2 may change how arcs are divided or drop collinear vertices while remaining topologically equal with a Hausdorff distance of 0. Identity of coordinate sequences must be separated from equality of geometry.

A smaller file does not automatically lead to faster operations either. E2 shows that building TopoJSON is costly, and E3 shows that decoding includes fragment handling and coordinate conversion. The results of Wachs and Tzirita Zacharatou [10] also place data loading in the context of a specific reader, scale and geometry. This report adds a small reproducible check; it does not replicate that study's design or confirm a ranking for all of its workloads.

A format with bbox metadata does not necessarily have a complete spatial index. With GeoParquet, skipping row groups through metadata or statistics also depends on data layout and the reader [19]. E2 reads all data and does not measure bbox queries, column selection, peak memory or scanning from object storage. Hence the speed results here are not sufficient to confirm pruning effectiveness or cloud query costs.

Limitations and future work

The largest limitation is synthetic data with a regular structure, few attributes and many shared boundaries. It is useful for isolating the arc-sharing mechanism, but may favor compression more

than point data, lines, polygons with many holes, complex administrative boundaries or long string attributes. Three scales up to only 10,000 objects do not represent national data. Measurement with a warm cache, no fsync and no isolation from host noise also limits how the times can be interpreted.

In addition, the formats use different implementations, and the TopoJSON decoder in the test package serves only the generated Polygon case. It is not a conformance test suite for the full specification. The experiments do not cover 3D data, a time axis, null geometry, nested attributes, curves, geodetic error or complete metadata preservation for every format. Five repetitions support only descriptive statistics; they do not demonstrate statistically significant differences in other deployment environments.

The next step should add a real dataset that is permitted for use, and publish its source, geometry types, CRS and processing steps. With boundary data, holes, multipart polygons, gaps and overlaps can also be tested; for map services, HTTP, memory and display time should be measured in the same application. When testing GeoParquet at large scale, full reads, column selection and spatial queries should be separated, with codec and row group layout recorded. These extensions answer new questions rather than prolonging the current measurement without changing its goal.

8 Conclusion

Representing spatial objects is a chain of related decisions: choosing an entity or field model, choosing a geometric representation, organizing relations, then choosing a format and delivery path. The knowledge of raster, vector, half-planes, spaghetti, network and topology in Chapter 2 “Representation of Spatial Objects” [1, pp. 31–51] still provides the basis for understanding modern technologies. TIGER/Line illustrates the value of combining geometry and relations, and shows the need to distinguish a source database from the products exported from it.

The three experiments show that arc sharing supports consistent boundary editing but does not by itself fix gaps; that size and speed depend on scale, codec and pipeline; and that tiling helps reduce the data needed for a viewport but comes with clipping, quantization and over-fetching costs. Accordingly, selection criteria should start from the information to be preserved and the operations to be performed. The package of code, data and per-repetition results allows these conclusions to be checked and rerun with other data, software or environments.

References

[1] Rigaux, Philippe; Scholl, Michel; Voisard, Agnès. “Representation of Spatial Objects”. Chapter 2 in Spatial Databases: With Application to GIS. Morgan Kaufmann, 2002, pp. 29–67 (printed page numbers of the book). https://doi.org/10.1016/B978-1-55860-588-6.X5000-3

[2] Duckham, Matt; Sun, Qian (Chayn); Worboys, Michael F.. GIS: A Computing Perspective. Edition 3. CRC Press, 2024. Checked against the open-access chapter on spatial concepts. Accessed November 2, 2026. https://doi.org/10.1201/9780429168093

[3] Goodchild, Michael F.; Yuan, May; Cova, Thomas J.. Towards a General Theory of Geographic Representation in GIS. International Journal of Geographical Information Science, 2007, 21(3), 239–260. https://doi.org/10.1080/13658810600965271

[4] Egenhofer, Max J.; Franzosa, Robert D.. Point-Set Topological Spatial Relations. International Journal of Geographical Information Systems, 1991, 5(2), 161–174. https://doi.org/10.1080/02693799108927841

[5] Günther, Oliver; Wong, Eugene. A Dual Space Representation for Geometric Data. Proceedings of the 13th International Conference on Very Large Data Bases, 1987, pp. 501–506. https://www.vldb.org/conf/1987/P501.PDF

[6] Güting, Ralf Hartmut; Schneider, Markus. Realms: A Foundation for Spatial Data Types in Database Systems. Advances in Spatial Databases (SSD 1993), 1993, vol. 692, pp. 14–35. https://doi.org/10.1007/3-540-56869-7_2

[7] Douglas, David H.; Peucker, Thomas K.. Algorithms for the Reduction of the Number of Points Required to Represent a Digitized Line or Its Caricature. The Canadian Cartographer, 1973, 10(2), 112–122. https://doi.org/10.3138/FM57-6770-U75U-7727

[8] De Floriani, Leila; Puppo, Enrico. Hierarchical Triangulation for Multiresolution Surface Description. ACM Transactions on Graphics, 1995, 14(4), 363–411. https://doi.org/10.1145/225294.225297

[9] Muller, David E.; Preparata, Franco P.. Finding the Intersection of Two Convex Polyhedra. Theoretical Computer Science, 1978, 7(2), 217–236. https://doi.org/10.1016/0304-3975(78)90051-8

[10] Wachs, Aske; Tzirita Zacharatou, Eleni. Analysis of Geospatial Data Loading. Proceedings of the Tenth International Workshop on Testing Database Systems (DBTest '24), 2024, pp. 36–42. https://doi.org/10.1145/3662165.3662761

[11] Netek, Rostislav; Masopust, Jan; Pavlicek, Frantisek; Pechanec, Vilem. Performance Testing on Vector vs. Raster Map Tiles---Comparative Study on Load Metrics. ISPRS International Journal of Geo-Information, 2020, 9(2), 101. https://doi.org/10.3390/ijgi9020101

[12] Balla, Dániel; Gede, Mátyás. Vector Data Rendering Performance Analysis of Open-Source Web Mapping Libraries. ISPRS International Journal of Geo-Information, 2025, 14(9), 336. https://doi.org/10.3390/ijgi14090336

[13] Li, Lin; Hu, Wei; Zhu, Haihong; Li, You; Zhang, Hang. Tiled Vector Data Model for the Geographical Features of Symbolized Maps. PLOS ONE, 2017, 12(5), e0176387. https://doi.org/10.1371/journal.pone.0176387

[14] Open Geospatial Consortium. OpenGIS Implementation Standard for Geographic Information -- Simple Feature Access -- Part 1: Common Architecture. Document 06-103r4. Version 1.2.1. 2011. Accessed November 2, 2026. https://www.ogc.org/standards/sfa/

[15] Butler, Howard; Daly, Martin; Doyle, Allan; Gillies, Sean; Hagen, Stefan; Schaub, Tim. The GeoJSON Format. Document 7946. 2016. Accessed November 2, 2026. https://doi.org/10.17487/RFC7946

[16] Bostock, Mike; Metcalf, Calvin. The TopoJSON Format Specification. No publication year assigned because the source does not state one. Accessed November 2, 2026. https://github.com/topojson/topojson-specification

[17] Open Geospatial Consortium. OGC KML 2.3. Document 12-007r2. 2015. Accessed November 2, 2026. https://docs.ogc.org/is/12-007r2/12-007r2.html

[18] Mapbox. Mapbox Vector Tile Specification. Version 2.1. 2016. Accessed November 2, 2026. https://mapbox.github.io/vector-tile-spec/

[19] GeoParquet contributors. GeoParquet Specification. Version 1.1.0. No publication year assigned because the source does not state one. Accessed November 2, 2026. https://geoparquet.org/releases/v1.1.0/

[20] Open Geospatial Consortium. OGC API -- Tiles -- Part 1: Core. Document 20-057. Version 1.0. 2022. Accessed November 2, 2026. https://docs.ogc.org/is/20-057/20-057.html

[21] U.S. Census Bureau. 2026 TIGER/Line Shapefiles Technical Documentation. 2026. The official technical contents page was used; this does not claim access to the complete technical PDF. Accessed November 2, 2026. https://www.census.gov/programs-surveys/geography/technical-documentation/complete-technical-documentation/tiger-geo-line.2026.html

[22] U.S. Census Bureau. TIGER Data Products Guide. Official guide page for TIGER products. Accessed November 2, 2026. https://www.census.gov/programs-surveys/geography/guidance/tiger-data-products-guide.html

Appendix A Reproduction guide

Extract the package Spatial_Representation_Sources_and_Data.zip. The experiments directory holds run_experiments.py, requirements.txt, README.md and the results of the run used in the report. No GIS service account is needed to run the synthetic dataset. Install the dependencies in a separate Python environment, then run from the experiments directory with the commands below.

Commands to rerun the experiments

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install -r requirements.txt
python run_experiments.py --output results_reproduced --repeats 5
```

On Windows, use the corresponding activation command .venv\Scripts\activate. Do not overwrite the original results directory if comparison is intended. Geometry and configuration are generated deterministically; times may vary with the machine, library versions and system load. When reporting a new run, keep the environment.json file and the measurement rows of that run, rather than replacing only the summary figures in the tables.

The file e1_topology.json holds the four shared-boundary scenarios; e2_raw_timings.csv holds 75 write and read repetitions, and e3_raw_timings.csv holds 15 full-domain and viewport decoding repetitions. The e2_summary and e3_summary files come in both CSV and JSON. The results/data directory holds the actual files of each format and the PBF tiles. Detailed geometry checks and metadata fields are kept in JSON; CSV serves filtering and tabulation.

The code theory_examples.py in the root directory checks the three states of half-plane classification, the invariance under multiplying the coefficients by the same positive number, and the edge table of the two squares. The run accompanying the report achieved 60 invariance comparisons, obtaining 6 vertices, 7 edges, left face A and right face B on the shared edge; with the outer face, $V - E + F$ equals 2. The files figures/draw_theory_figures.py and figures/draw_result_figures.py rebuild the diagrams and charts.

The sources directory stores the structured content, citation list and DOCX build program. The report's reference numbers are mapped in bibliography_report.json; these numbers should not be equated with the R codes of the previously downloaded reference PDF package. Rebuilding the DOCX/PDF requires additional publishing tools, separate from the GIS dependencies used to run the experiments.

Appendix B Mapping to the evaluation criteria

Table 12 Content addressing the provided evaluation framework

Criterion	Evidence in the report	Maximum points per the assignment
Core theory	Section 3; three original figures; two pieces of pseudocode; correctness conditions and verified examples	3.0
Updates and experiments	Sections 4–7; five formats; three experiments actually run; code and raw data	3.0
Scientific structure	Abstract, research questions, related work, method, results, discussion, conclusion	2.0
References	Original book, original papers and official specifications; numbered citations with DOI or URL	1.0
Timeliness	Submission dates of the outline, report and revised version must be checked against the instructor's deadlines	1.0

The mapping table only points to evidence and the assignment's maximum points; it does not self-grade or confirm on-time submission. Before submitting, the group must add member information according to the class template, check the page and deadline requirements, and rerun the experiments in the group's environment if the instructor requires direct evidence of installation and execution.